

2

The Game Design Document

Before we start typing a single line of code, we need to complete a crucial step in a game's development cycle, which is creating a **Game Design Document (GDD)**. A GDD is a blueprint of our entire project; its primary purpose is to put on paper the overall vision of our game's core concepts and act as a guide to a multidisciplinary team of developers during the arduous journey of a lengthy production cycle.

A GDD can include detailed descriptions of the following elements:

- Lists of core visual, animation, and audio ingredients
- Synopsis, character biographies, and narrative structures
- Marketing research material and monetization strategies
- Descriptions and diagrams that illustrate systems and mechanics

In most cases, a game designer is responsible for writing and maintaining the GDD, while a game programmer is responsible for implementing the described systems and mechanics in the document. Therefore, programmers and designers must give each other feedback throughout the entire process of formulating the GDD. If both sides fail to work together, the game designer will write up systems that are impossible to implement within a reasonable timeframe. Or programmers will waste time writing code for game systems that are defective in their overall design.

Considering that this book's focus is limited to programming and not game design, I will present a simplified and short version of the GDD for the game project we will be working on throughout this book. *The document section will also include some tips and notes for those who are not familiar with the process of analyzing a GDD.*



For reasons of brevity, I did not mention the participation of artists, animators, and audio engineers in the process of drafting the GDD. But their participation is essential because they need to know how many assets they will need to produce to realize the overall vision of the game.

In this chapter, we will cover the following topics:

- An overview of the game's synopsis and core mechanics
- An explanation of the core game loop and objectives
- A list of the game's ingredients and systems

The design document

The following game design document is divided into parts representing the individual point of interest in the overall project. For example, the game's synopsis section might be of interest to those working on the game's narrative elements. At the same time, the parts about minimum requirements and systems are geared toward the programmers. But whatever our specialization, it's good practice to read the entire design document and gain a complete mental model of the project before working on it.

Game overview

Blade Racer (*working title*) is a futurist arcade racing game in which the player attempts to control a turbo-charged motorcycle across an obstacle course to get to the finish line with the highest score. The race track is built on a railway system in which the player can steer a vehicle from one rail to the other to avoid obstacles. Across the pathways, the track system spawns pickup items in strategic positions that reward players with speed and shield boosts. Only players with razor-sharp reflexes will cross the finish line with the fastest time and without damaging their bike.



A working title is a temporary name for a project used during production. It's not final and can be changed when the project is ready for release.

Unique selling points

The following are potential unique selling points:

- International leaderboard with which players worldwide can compete to get their name on the top of the list
- The most challenging racing game on the market
- The game can be played on any device, from mobile to PC



Unique selling points are good to have in a GDD if you work with a publisher. It showcases that you have a vision of the final product and how you will market it. Take note that the examples above are placeholders and not conclusive.

Minimum requirements

The following are the minimum requirements for the current target platforms:

For **Mobile**:

- **OS:** Android 10
- **Model:** Samsung S9

For **PC**:

- **OS:** Windows 10
- **Graphics card:** Nvidia GTX 980

* We will support the equivalent ATI card from the same generation of the specified GTX card.



Specifying minimum platform requirements right at the beginning of a project is very beneficial to game programmers. A platform with limited resources will demand more optimization of the visual assets and codebase for the game to run at a steady frame rate on all targeted devices.

Game synopsis

It's 2097. The human race has achieved mastery over the planet and now has set its sights on exploring beyond the edge of the solar system in the hope of facing new challenges and discovering new worlds. As peace prevails globally, those who have not left the planet experience a new form of collective boredom. Thanks to technology and biohacking, all the challenges of the past are gone. Anyone can be a top athlete, beautiful, and talented without any struggle.

Sensing the growing unrest in the global population, Neo Lusk, an eccentric tech entrepreneur and quadrillionaire, decides to invent a new sport involving turbo-charged motorcycles that can reach speeds most humans have problems handling. To make it even more challenging, he designs a unique race track system involving rails and obstacles.

This new sport invigorates the global population, and millions decide to participate. But few can control these high-tech bikes and maneuver them through a deadly layout of obstacles. Only the genuinely gifted with nerves of steel and reflexes as sharp as a blade can master this sport, but at the moment, the world is waiting to know who will be the first to be crowned its champion.



Even though our current project is an arcade racing game, which is a genre that's usually very light on story but heavy on gameplay, it's still good to have some narrative elements that provide some context to the game's world. It triggers the imagination of the player and gives an overall meaning to the game. Even classic arcade games such as Pac-Man and Missile Command have a certain degree of narrative ingredients that produce a feeling that you are playing a character inside a virtual world.

Game objectives

The list of core objectives (*a.k.a. player goals*) are the following:

- The main objective (*a.k.a. win state*) is to cross the finish line without any fatal crashes.
- The secondary objective is to get the best score by earning "risk" points by taking risky racing lines when swerving between obstacles.
- As a bonus objective, a player can get extra points by picking up a specific amount of collectible items (*a.k.a. pickups*) that are spawned across the tracks. *This objective is aimed at "completionists."*
- The ultimate objective of the game is to get the highest score while finishing the race with the fastest time.



A good game designer will always design several objectives to satisfy the various kinds of players to keep them engaged as long as possible. For example, a player with a "completionist" mindset doesn't simply want to finish a game but complete it by attaining every possible objective and getting every collectible item.

The player can fail (*a.k.a. fail state*) at achieving the core objectives with the following actions:

- The main "fail state" is triggered when a player doesn't arrive at the finish line because of a fatal crash.
- If the checkpoint game mode is activated, the player can fail by not crossing each checkpoint in time.



It's good practice to also clearly define all possible "fail conditions" of a game early on because it helps analyze how many distinct wins and fail state conditions we will need to handle.

Game rules

The game level is an open environment where the player controls a motorcycle that runs at high speed on a rail system composed of four tracks. The player can navigate the vehicle sideways to avoid obstacles or capture "pickup" items spawned along the tracks. The player must race to the finish line while reaching the checkpoints that are set along the paths within the minimum required time while avoiding damaging the vehicle by steering clear of obstacles. The player can score points by choosing risky racing lines such as turning to avoid an impediment within inches of colliding with it. The player can also get bonus points by capturing a specific number of "pickup" items in the correct order.

Game loop

The following diagram illustrates the core game loop:

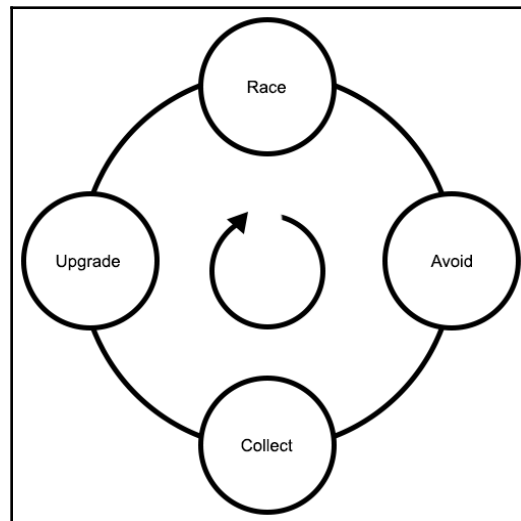


Figure 2.1 – Diagram of the core game loop

The game has four pillars in its core game loop:

- **Race:** The player races against the clock toward the finish line.
- **Avoid:** The player must avoid obstacles while going as fast as possible while taking risks to gain points.
- **Collect:** The player can pick up items and rewards placed on the race track during the race.
- **Upgrade:** During or after the race, the player can upgrade the vehicle with the collected items.

Game environment

The primary setting for each level is a futuristic deserted space with four rails that span the horizon. There are metallic light polls and barriers on the side of the railway. The sky is cloudy, and a constant fog obscures the skyline while rainstorms often occur. On the tracks, there are various obstacles and items for the player to avoid or capture. The level design is focused on punishing slow players and rewarding those that go fast and take risks.

Camera, control, character (3Cs)

3Cs (**camera**, **control**, and **character**) is an industry-standard term that is used in the AAA industry in studios such as *Ubisoft*. It basically states that the core of a player's experience is defined by the balance between the harmonious relationship of the camera, the player character, and the control scheme. If a player feels that the character responds to inputs as expected while the camera seamlessly positions itself at an optimal angle, then the overall experience will be more immersive. Let's look at the 3Cs in detail.

Camera

The main camera will follow the vehicle from the rear in a third-person view. It will automatically position and rotate itself on the nearest track opposite the vehicle's direction during a turn. During a turbo boost, the camera will pull back and start shaking slightly to give the sense of having difficulty following the vehicle as it speeds away. And when the motorbike slows back down to normal speed, the distance between the camera and motorcycle will snap back to its default distance, as we can see in the following diagram:

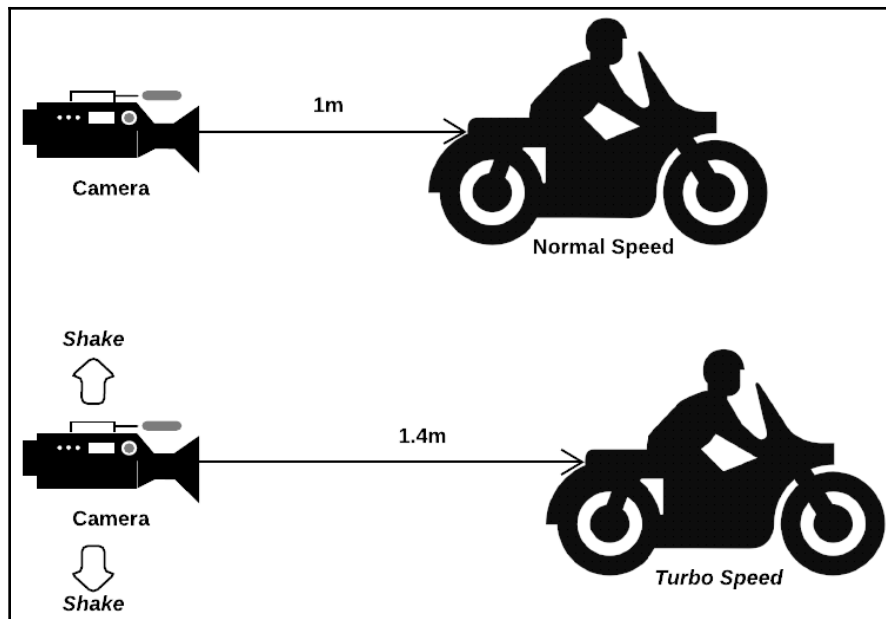


Figure 2.2 – Illustration of the camera positions

In this current iteration of the game, there will be no first-person or rear-view camera.

Character

In this section, we will define the main player-controlled character.

Character description

The main playable character is a human pilot that drives a futurist motorcycle vehicle that runs on a railway system and is limited to maneuvering from one rail to another at very high speeds. Considering that the main playable character is inside the vehicle's cockpit at all times, we can consider the vehicle and the pilot as one playable character during gameplay segments. The game will not have any other playable humanoid characters or types of controllable vehicles.

Character metrics

The following section outlines the primary character/vehicle metrics:

- **Min Speed:** 500 km/h
The vehicle is always moving during the race and can never completely stop.
- **Max Speed:** 6,500 km/h
The vehicle can only stay at max speed for a couple of seconds and its handling capacity will reduce to its minimum amount.
- **Max Health:** 100%
The vehicle has a damage meter calculated as a percentage.
- **Attack Damage:** Depends on the weapon.
The vehicle has no default weapon system but has an attachment slot for upgrades.
- **Handling:** Depends on the current speed.
The vehicle's handling capacity is reduced as it reaches its top speed.

Character states

The following section outlines the primary character/vehicle states:

- **Idle:** The idle state of the vehicle is an animation cycle that shows the chassis vibrating from the roaring engine and the taillights blinking.
- **Speeding:** The speeding state has an animation cycle of the wheels turning.
- **Slowing:** The slowing animation slows down the wheels and flashes the taillights.
- **Turning:** The turning state has an animation of the vehicle tilting and moving toward the direction of the nearest track.
- **Braking:** The braking animation will make the vehicle tilt and slide sideways on a forward trajectory. This state is activated only once the player crosses the finishing line.
- **Shaking:** The shaking animation rises in intensity as the handling capacity of the vehicle is reduced at top speeds.
- **Flipping:** The flipping animation flips the vehicle forward when it collides with a roadblock obstacle.
- **Sliding:** The sliding animation turns the vehicle sideways and slides in a forward direction.



It's important right from the start that the design and programming team define each type of playable controllable character, device, or vehicle in the game. Mainly because implementing 3Cs for a car or a two-legged humanoid character demands different approaches. Like in the real world, driving and walking are different types of experiences with specific mechanical intricacies.

Controller

The following section outlines the primary controller scheme, which is the keyboard:

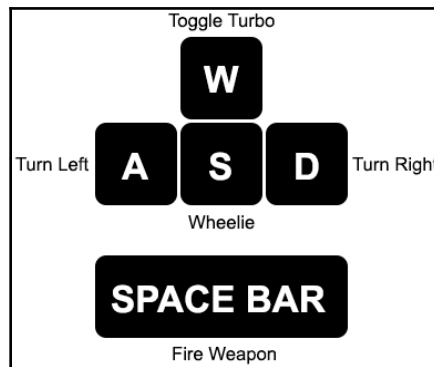


Figure 2.3 – Diagram of the keyboard input keys



We will only support the keyboard as the primary input device for this book's game project for reasons of simplicity.

Game ingredients

The following section outlines the core game ingredients:

- **Bikes** are the main vehicle of the game and can be controlled by the player or **AI (Artificial Intelligence)**.
- **Pickups** are items that are spawned on the tracks and that the player can pick up by colliding with them. Each type of item will have a distinct shape and will float a couple of inches above the track.

- **Obstacles** are environment ingredients that are spawned on the tracks. Certain types of obstacle entities can damage the vehicle while others block the vehicle and cause it to crash.
- **Weapons** are ingredients that can be attached to an available slot on the player's vehicle. The player's weapon ingredients are laser-based and permit them to destroy obstacles.
- **Drones** are robotic entities that fly around in flight patterns and attack the player with laser beams.

Superbikes

Name	Description	Upgrade Slots	Handling	Top Speed
Venom	The entry-level model for beginner racers.	1	80%	1,900 km/h
Silver Bullet	The advanced model for intermediate racers.	2	60%	4,000 km/h
Death Rider	The most advanced model, only accessible to top players.	4	40%	6,500 km/h



The bike's handling is defined by how quickly the vehicle reacts to the player inputs. As the speed increases, the overall stability decreases. Therefore, the bike will start sliding while turning and will take longer to stabilize and respond to the player's commands.

Pickups

Name	Description	Effect	Value
Collectible	A collectible item is spawned on the length of a track in numbered order.	Increments the risk score	10 risk points
Turbo Boost	This item is spawned on the tracks at specified locations.	Fills up the turbo meter	10% turbo boost
Damage Repair * a.k.a. Health Restore	This item is spawned on the tracks at specific locations.	Repairs damage	10% of heath

Obstacles

Name	Description	Effect	Damage
Track Barrier	A tall and flat barrier that blocks the track but breaks on impact.	Causes damage but doesn't stop the vehicle	20% of health
Road Block	A short barricade stops the vehicle by making it flip forward and crash.	Stops the vehicle	100% of health

Weaponry

Name	Description	Effect	Range
Laser Shooter	Singular ray cast weapon	Destroys everything within range	10 meters
Plasma Blaster	Multiple ray cast weapons	Destroys everything within range and a field of view of 60 degrees	20 meters

Game systems

The following section outlines the core game systems:

- **Rail system**

The rail system permits the player's vehicle to navigate sideways on four individual tracks. Each track can spawn obstacles and pickups along its length.

- **Risk system**

The risk system rewards a player that takes risks, for example, by swerving away from obstacles in the nick of time at top speed. The number of points given is based on the distance between the vehicle and the barrier when the player avoided it. A sensor installed in front of the bike with a range of 5 meters is constantly monitoring for any obstructions on the path of the current track. Once a potential collision is detected, the sensor calculates the distance between the obstacles and the vehicle. Points are determined based on the distance between the player and when the hindrance was avoided.

The following diagram illustrates the forward-facing sensor:

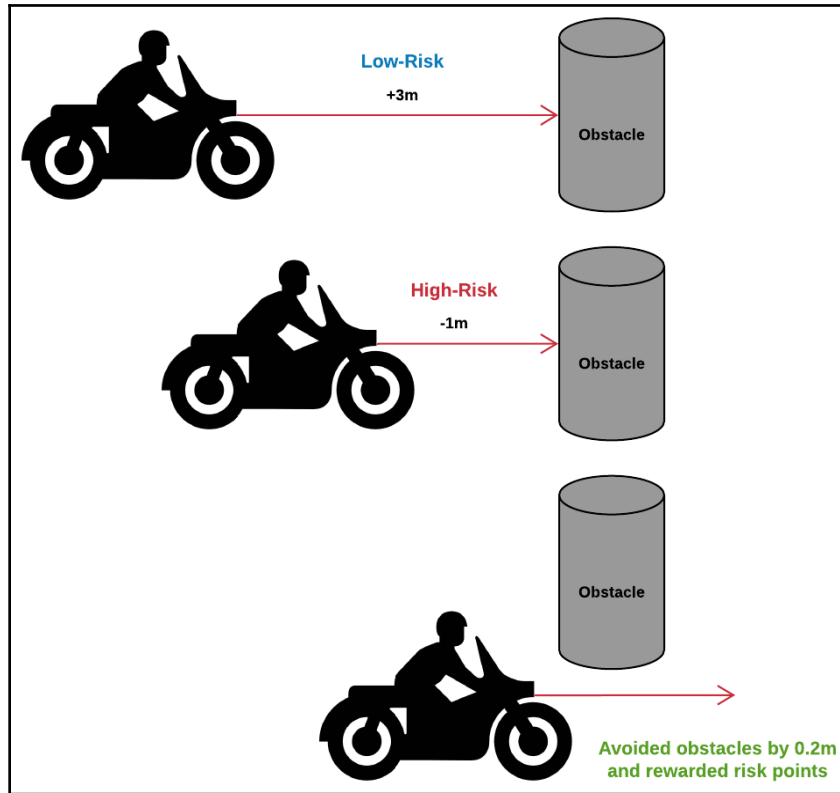


Figure 2.4 – Diagram of the forward-facing sensor detecting obstacles

- **Turbo Boost System**

The **Turbo Boost System (TBS)** permits the vehicle to reach its top speeds. To activate the system, the player must fill up the turbo meter by picking up turbo boost items. The TBS system has a gauge on the **Head-Up Display (HUD)** that permits the player to visualize the amount of turbo boost available. When the TBS is engaged, the vehicle handling is lessened and it is more vulnerable to damage.

- **Vehicle upgrade system**

The player's vehicle has item slots that permit them to attach weapons and other upgrades. Depending on the type of vehicle, the number of slots will vary.



The difference between a game mechanic and a system is sometimes subject to confusion. In this book, I will define a game system as a collection of game mechanics and ingredients. For example, weaponry pickups are ingredients, but the actions of picking them up on the battlefield and then selecting the right weapon in the inventory menu to win a battle are individual mechanics. But all these elements are distinct components of an entire weapon system.

Game menu

In this section, we'll outline the game's menus:

Main Menu: The main menu will be accessible at the start of the game. The menu options are as follows:

- **Start Race:** This option loads the race track scene.
- **Scores:** This option displays the latest top 10 scores.
- **Exit:** This option closes the game's window.

In-Game Menu: The in-game menu is only accessible during the race. The menu options are as follows:

- **Restart Race:** Restarts the race from the beginning
- **Exit Race:** Returns the player back to the lobby scene

Game HUD

The following section outlines the game's **Head-up Display (HUD)**:

The HUD will only be displayed during the race and will include the following interface components:

- **Turbo Meter**
- **Speed Meter**
- **Damage Meter**
- **Countdown Timer**
- **Track Progress Bar**



Often, the final design of a game's menus and HUDs is completed in later stages of production. Mostly because the overall design, art style, and structure of a game can change several times during the earlier stages of production, therefore, the flow of the menus and the layout of the components of the HUDs are subject to constant change.

Summary

In this chapter, we took the time to review the current draft of the design document of the book's game project. This draft of the GDD might not be complete or perfect, but often, game projects start with incomplete design documentation and ideas. So it's still good that we took the time to review it. We will not implement every system, mechanic, and ingredient described in this book because our primary goal is to learn design patterns by building a game, not necessarily a complete one.

Even though game design is not the main subject of this book, learning to think like a designer is a skill that can only help anyone become a better game programmer. Reading design documentation helps us understand the game's vision, structure, mechanics, and scope. And these are things to keep in mind when making software architecture decisions.

The next chapter includes a short primer on basic Unity programming fundamentals. If you are already an experienced programmer, you can skip to the hands-on section of the book. But if you are a beginner, I would recommend reading the primer before implementing the upcoming chapters' design patterns.

Further reading

For more information, you can refer to the following books:

- *The Art of Game Design* by Jesse Schell
- *Game Design: Secrets of the Sages* by Marc Saltzman
- *Level Up! The Guide to Great Video Game Design* by Scott Rogers
- *Rules of Play: Game Design Fundamentals* by Katie Salen and Eric Zimmerman